

4.8 节练习

练习 4.25: 如果一台机器上 `int` 占 32 位、`char` 占 8 位，用的是 Latin-1 字符集，其中字符 'q' 的二进制形式是 01110001，那么表达式 `~'q' << 6` 的值是什么？

练习 4.26: 在本节关于测验成绩的例子中，如果使用 `unsigned int` 作为 `quiz1` 的类型会发生什么情况？

练习 4.27: 下列表达式的结果是什么？

```
unsigned long ul1 = 3, ul2 = 7;
```

(a) `ul1 & ul2`

(b) `ul1 | ul2`

(c) `ul1 && ul2`

(d) `ul1 || ul2`

4.9 sizeof 运算符

`sizeof` 运算符返回一条表达式或一个类型名字所占的字节数。`sizeof` 运算符满足右结合律，其所得的值是一个 `size_t` 类型（参见 3.5.2 节，第 103 页）的常量表达式（参见 2.4.4 节，第 58 页）。运算符的运算对象有两种形式：

```
sizeof (type)
sizeof expr
```

在第二种形式中，`sizeof` 返回的是表达式结果类型的大小。与众不同的一点是，`sizeof` 并不实际计算其运算对象的值：

```
Sales_data data, *p;
sizeof(Sales_data);    // 存储 Sales_data 类型的对象所占的空间大小
sizeof data;          // data 的类型的大小，即 sizeof(Sales_data)
sizeof p;              // 指针所占的空间大小
sizeof *p;             // p 所指类型的空间大小，即 sizeof(Sales_data)
sizeof data.revenue;   // Sales_data 的 revenue 成员对应类型的大小
sizeof Sales_data::revenue; // 另一种获取 revenue 大小的方式
```

157

这些例子中最有趣的一个是 `sizeof *p`。首先，因为 `sizeof` 满足右结合律并且与 `*` 运算符的优先级一样，所以表达式按照从右向左的顺序组合。也就是说，它等价于 `sizeof(*p)`。其次，因为 `sizeof` 不会实际求运算对象的值，所以即使 `p` 是一个无效（即未初始化）的指针（参见 2.3.2 节，第 47 页）也不会有什么影响。在 `sizeof` 的运算对象中解引用一个无效指针仍然是一种安全的行为，因为指针实际上并没有被真正使用。`sizeof` 不需要真的解引用指针也能知道它所指对象的类型。

C++11 新标准允许我们使用作用域运算符来获取类成员的大小。通常情况下只有通过类的对象才能访问到类的成员，但是 `sizeof` 运算符无须我们提供一个具体的对象，因为要想知道类成员的大小无须真的获取该成员。

C++
11

`sizeof` 运算符的结果部分地依赖于其作用的类型：

- 对 `char` 或者类型为 `char` 的表达式执行 `sizeof` 运算，结果得 1。
- 对引用类型执行 `sizeof` 运算得到被引用对象所占空间的大小。
- 对指针执行 `sizeof` 运算得到指针本身所占空间的大小。
- 对解引用指针执行 `sizeof` 运算得到指针指向的对象所占空间的大小，指针不需有效。

- 对数组执行 `sizeof` 运算得到整个数组所占空间的大小，等价于对数组中所有的元素各执行一次 `sizeof` 运算并将所得结果求和。注意，`sizeof` 运算不会把数组转换成指针来处理。
- 对 `string` 对象或 `vector` 对象执行 `sizeof` 运算只返回该类型固定部分的大小，不会计算对象中的元素占用了多少空间。

因为执行 `sizeof` 运算能得到整个数组的大小，所以可以用数组的大小除以单个元素的大小得到数组中元素的个数：

```
// sizeof(ia)/sizeof(*ia)返回 ia 的元素数量
constexpr size_t sz = sizeof(ia)/sizeof(*ia);
int arr2[sz];    // 正确: sizeof 返回一个常量表达式, 参见 2.4.4 节 (第 58 页)
```

因为 `sizeof` 的返回值是一个常量表达式，所以我们可以用 `sizeof` 的结果声明数组的维度。

4.9 节练习

练习 4.28: 编写一段程序，输出每一种内置类型所占空间的大小。

练习 4.29: 推断下面代码的输出结果并说明理由。实际运行这段程序，结果和你想象的一样吗？如果不一样，为什么？

```
int x[10]; int *p = x;
cout << sizeof(x)/sizeof(*x) << endl;
cout << sizeof(p)/sizeof(*p) << endl;
```

练习 4.30: 根据 4.12 节中的表（第 147 页），在下述表达式的适当位置加上括号，使得加上括号之后表达式的含义与原来的含义相同。

- | | |
|----------------------------------|--------------------------------------|
| (a) <code>sizeof x + y</code> | (b) <code>sizeof p->mem[i]</code> |
| (c) <code>sizeof a < b</code> | (d) <code>sizeof f()</code> |

4.10 逗号运算符

逗号运算符（comma operator）含有两个运算对象，按照从左向右的顺序依次求值。和逻辑与、逻辑或以及条件运算符一样，逗号运算符也规定了运算对象求值的顺序。

158

对于逗号运算符来说，首先对左侧的表达式求值，然后将求值结果丢弃掉。逗号运算符真正的结果是右侧表达式的值。如果右侧运算对象是左值，那么最终的求值结果也是左值。

逗号运算符经常被用在 `for` 循环当中：

```
vector<int>::size_type cnt = ivec.size();
// 将把从 size 到 1 的值赋给 ivec 的元素
for(vector<int>::size_type ix = 0;
    ix != ivec.size(); ++ix, --cnt)
    ivec[ix] = cnt;
```

这个循环在 `for` 语句的表达式中递增 `ix`、递减 `cnt`，每次循环迭代 `ix` 和 `cnt` 相应改变。只要 `ix` 满足条件，我们就把当前元素设成 `cnt` 的当前值。